

The Maths Behind XGBoost

Contents

1. Overview	2
2. A boosting model is a sum of functions	3
3. What we must find: the leaf weights	3
4. The objective: loss plus a penalty on the tree	4
5. The objective, stage by stage	4
6. Why we cannot simply differentiate	5
7. The Taylor series	6
8. The second-order approximation of the objective	7
9. From a sum over rows to a sum over leaves	7
10. The best leaf output	9
11. Regression: gradient and Hessian of squared error	9
12. Classification: gradient and Hessian of log loss	10
13. The best objective, and the similarity score	11
14. The gain of a split	12
15. Summary	14
16. Sources	14
17. Key terms	14

Where this fits: Pipeline map steps 0 (Foundations), 8 (Model). See the [Course map](#).



- **Builds on:** Binning and binarization ([Note 32](#)); Eigenvectors and eigenvalues ([Note 48](#)); Derivatives of one variable ([Note 51](#)); Regularisation ([Note 63](#)); Gradient boosting ([Note 122](#)); Missing values ([Note 123](#)).
- **Leads to:** Convex and non-convex loss ([Note 590](#)).

1. Overview

Key point: XGBoost's formulas all come from one calculation: write the objective (loss + a penalty on the new tree), approximate it with a second-order Taylor series, group the rows by leaf, and minimise. The minimum gives the leaf output; its value gives the similarity score and the gain.

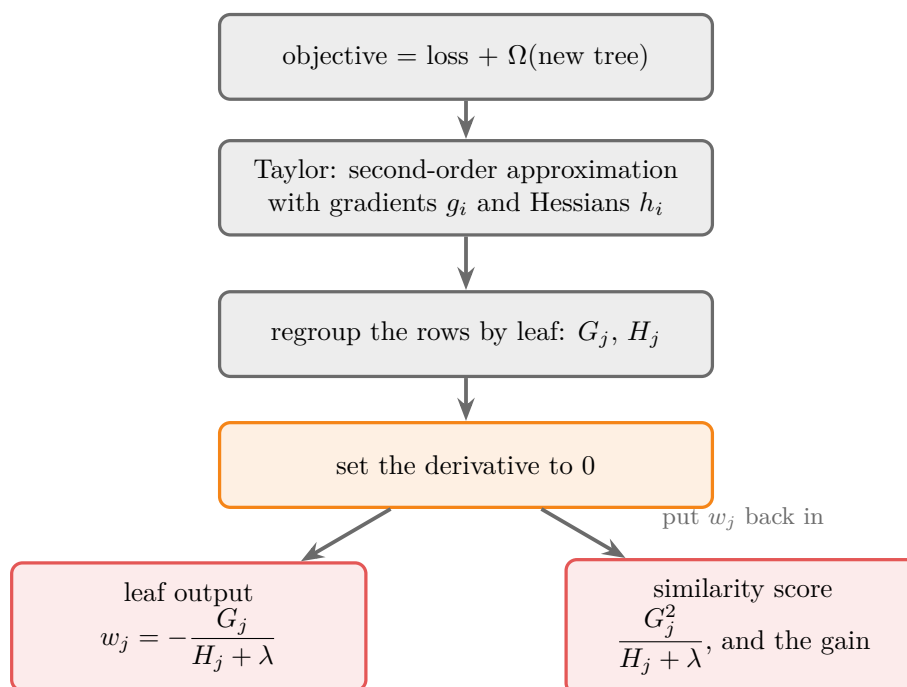


Figure 1: The derivation in five steps: from the objective to the leaf output, the similarity score and the gain

Two earlier Notes, on [XGBoost regression](#) and [XGBoost classification](#), used four formulas without proof:

	Regression	Classification
Similarity score	$(\sum r_i)^2 / (n + \lambda)$	$(\sum r_i)^2 / (\sum p_i(1 - p_i) + \lambda)$
Output value	$\sum r_i / (n + \lambda)$	$\sum r_i / (\sum p_i(1 - p_i) + \lambda)$

This Note derives all four, following the XGBoost paper (Chen and Guestrin 2016, §2). Figure 1 is the plan. The maths needs derivatives and the chain rule ([gradient descent Note](#)) and the gradient boosting algorithm ([gradient boosting maths Note](#)).

2. A boosting model is a sum of functions

Key point: The prediction is the base model plus the output of every tree. Each tree is a function of the input, so the whole model is a sum of functions.

Take students with CGPA, IQ and a package. The base model is the mean, say 14.5 LPA. The second model is a tree trained on the residuals; for a student with CGPA 9 and IQ 102 it lands in a leaf with output 2.1. The prediction is $14.5 + 2.1 = 16.6$.

Every model here is a function: input in, number out. A tree computes its number with if-else questions, but it is still a function. Calling the models $f_1, f_2, \dots, f_t$, the prediction for row i is

$$\hat{y}_i = f_1(x_i) + f_2(x_i) + \dots + f_t(x_i) = \sum_{k=1}^t f_k(x_i)$$

This is the additive model of the [gradient boosting maths Note](#). (The learning rate is left out of the maths; it is applied afterwards, when the tree is added.)

3. What we must find: the leaf weights

Key point: Once the newest tree's splits are fixed, the only unknowns are its leaf outputs $w_1, \dots, w_T$. We want the values that make the loss smallest.

A tree with T leaves gives one of T numbers, depending on the leaf a row lands in. These leaf outputs are also called **leaf weights**, written $w_1, w_2, \dots, w_T$. A tree with 3 leaves has three unknowns, not one.

Good leaf weights bring each prediction $\hat{y}_i$ close to the true value y_i . The gap between them is measured by a loss function $L(y_i, \hat{y}_i)$, such as squared error ([regression metrics Note](#)). So the task is an optimisation problem: choose the w_j that make the total loss smallest.

4. The objective: loss plus a penalty on the tree

Key point: Gradient boosting minimises the loss alone. XGBoost minimises the loss plus $\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum w_j^2$, which punishes trees with many leaves and with large leaf outputs.

Gradient boosting accepts any differentiable loss and minimises $\sum_i L(y_i, \hat{y}_i)$. XGBoost adds a **regularisation term** Ω for the newest tree. The sum of the two is called the **objective function**: loss plus regularisation.

1. **In words:** the total loss over all rows, plus γ times the number of leaves, plus half of λ times the sum of the squared leaf weights.

2. **Formula:**

$$\text{Obj} = \sum_{i=1}^n L(y_i, \hat{y}_i) + \Omega(f), \quad \Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^T w_j^2$$

3. **Example:** the first tree of the [XGBoost regression Note](#) has $T = 3$ leaves with outputs 0.625, -2.125 and 3.625 . With $\gamma = 1$ and $\lambda = 1$:

$$\Omega = 1 \times 3 + \frac{1}{2} \times 1 \times (0.625^2 + 2.125^2 + 3.625^2) = 3 + \frac{1}{2} \times 18.05 = 12.02$$

Both parameters are hyperparameters that we choose. A larger γ makes every extra leaf cost more; a larger λ makes large outputs cost more, as the L2 penalty does in ridge regression ([ridge regression maths Note](#)).

Extra: This γ is a penalty per leaf. It is not the γ_{jm} used for leaf values in the [gradient boosting maths Note](#); the two papers just use the same Greek letter. In this Note leaf values are always w_j .

5. The objective, stage by stage

Key point: At stage t everything before is fixed; only the new tree f_t is unknown. The objective is the loss of (old prediction + $f_t(x_i)$) plus $\Omega(f_t)$.

Write the objective at each stage. The base model f_1 is a constant, not a tree, so it gets no Ω .

- **Stage 1** (base model only): $\text{Obj}^{(1)} = \sum_i L(y_i, f_1(x_i))$
- **Stage 2:** $\text{Obj}^{(2)} = \sum_i L(y_i, f_1(x_i) + f_2(x_i)) + \Omega(f_2)$
- **Stage 3:** $\text{Obj}^{(3)} = \sum_i L(y_i, f_1(x_i) + f_2(x_i) + f_3(x_i)) + \Omega(f_3)$

Only the newest tree gets a penalty: the earlier trees were already penalised at their own stages and are now fixed.

At a general stage t , everything up to f_{t-1} is already known. We call that sum $\hat{y}_i^{(t-1)}$, the prediction after $t - 1$ stages.

1. **In words:** the loss of each row when the new tree's output is added to the old prediction, summed over the rows, plus the penalty on the new tree.

2. Formula:

$$\text{Obj}^{(t)} = \sum_{i=1}^n L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

3. **Example:** in the regression Note at stage 2, student 1 has $y_1 = 4.5$ and $\hat{y}_1^{(1)} = 7.375$. With squared error $L = \frac{1}{2}(y - \hat{y})^2$ its loss is $\frac{1}{2}(4.5 - 7.375 - f_2(x_1))^2$: small when $f_2(x_1)$ is close to -2.875 , the residual.

The goal is to choose the new tree, and so its leaf weights, to make $\text{Obj}^{(t)}$ as small as possible.

6. Why we cannot simply differentiate

Key point: For a general loss, the objective has no simple closed-form minimum in f_t . XGBoost replaces the loss by a simple quadratic that is easy to minimise.

In linear regression the prediction is $mx + b$, a smooth function of m and b . The loss is a smooth bowl, so we set its derivatives to 0 ([linear regression maths Note](#)) or walk downhill ([gradient descent Note](#)).

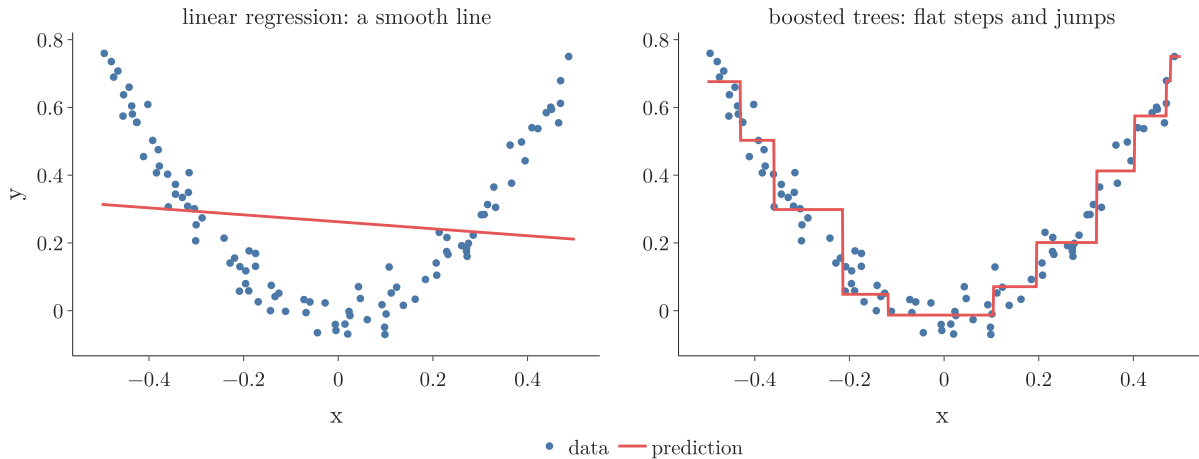


Figure 2: The same data fitted by a straight line (left) and by boosted trees (right): the trees give flat steps with sudden jumps

A boosted model is different. Its output is a staircase: flat inside each region, with jumps at the split points (Figure 2, right). The loss as a function of the trees is therefore not a smooth bowl. Even for one leaf, log loss is awkward: when the observations (rows) start from different log-odds, setting the derivative to 0 gives an equation with no simple formula for the answer (Extra in section 12).

The way out is to approximate the loss near the current prediction by something smooth and simple, a parabola, and minimise that instead. The tool for this is the Taylor series.

7. The Taylor series

Key point: Near a point a , any smooth function can be approximated by a polynomial built from its derivatives at a . More terms give a better approximation over a wider range.

The **Taylor series** approximates a complicated function by a polynomial. It uses the function's value and derivatives at one point a .

1. **In words:** start from the value at a , add the slope times the distance from a , add half the second derivative times the distance squared, and so on.

2. **Formula:**

$$f(x) \approx f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \dots$$

f' , f'' and f''' are the first, second and third derivatives; $2! = 2$ and $3! = 6$.

3. **Example:** $f(x) = e^x$ around $a = 0$. Every derivative of e^x is e^x , and $e^0 = 1$, so

$$e^x \approx 1 + x + \frac{x^2}{2} + \frac{x^3}{6}$$

At $x = 0.5$: $1 + 0.5 + 0.125 + 0.0208 = 1.6458$, against the true $e^{0.5} = 1.6487$.

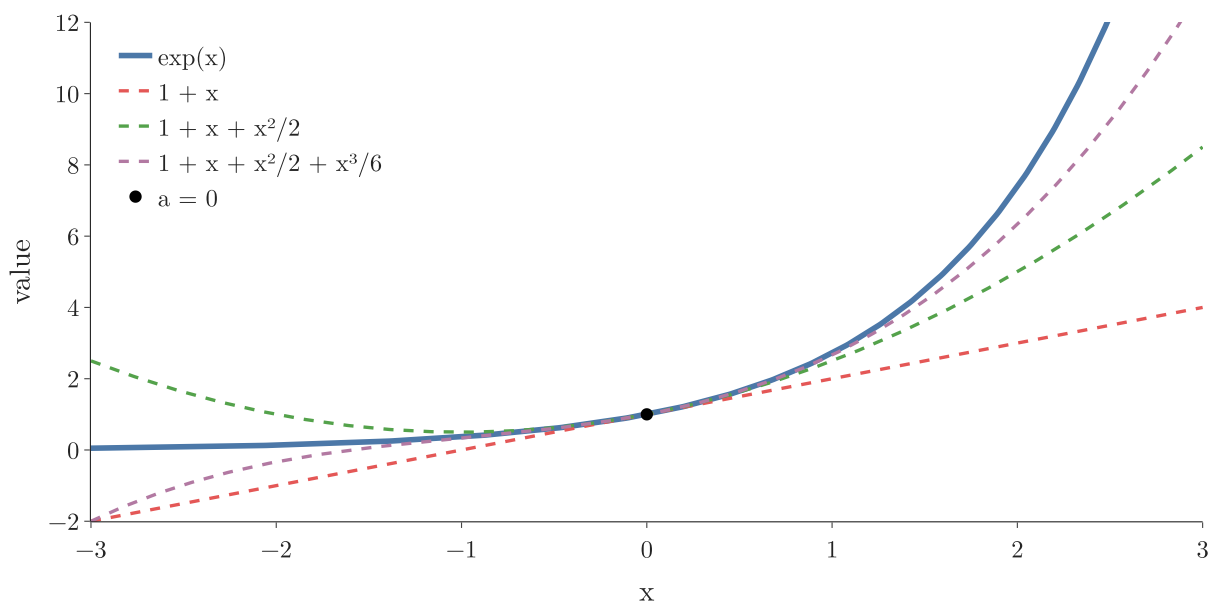


Figure 3: e^x and its Taylor approximations around 0: each extra term follows the curve over a wider range

Figure 3 shows the pattern. The straight line $1 + x$ is right only near 0. Adding $x^2/2$ follows the curve further, and adding $x^3/6$ further still. XGBoost stops at the second-order term, a parabola.

8. The second-order approximation of the objective

Key point: Expand each row's loss around the old prediction: $\text{loss} \approx \text{old loss} + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2$, with g_i the gradient and h_i the Hessian of the loss at the old prediction.

We apply the Taylor series to each row's loss $L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i))$, seen as a function of the prediction. The point we expand around is the old prediction, and the step away from it is the new tree's output:

- $x = \hat{y}_i^{(t-1)} + f_t(x_i)$ (the new prediction);
- $a = \hat{y}_i^{(t-1)}$ (the old prediction);
- $x - a = f_t(x_i)$ (the new tree's output).

Two names for the derivatives at the old prediction:

- the **gradient** g_i : the first derivative of the loss with respect to the prediction;
- the **Hessian** h_i : the second derivative.

$$g_i = \frac{\partial L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}, \quad h_i = \frac{\partial^2 L(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}$$

1. **In words:** each row's new loss is approximately its old loss, plus its gradient times the new tree's output, plus half its Hessian times that output squared.
2. **Formula:**

$$\text{Obj}^{(t)} \approx \sum_{i=1}^n \left[L(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

3. **Example:** squared error $L = \frac{1}{2}(y - \hat{y})^2$ gives $g_i = \hat{y}_i - y_i$ and $h_i = 1$ (section 11). For student 1, $g_1 = 7.375 - 4.5 = 2.875$, so with $f = f_t(x_1)$:

$$\frac{1}{2}(2.875)^2 + 2.875f + \frac{1}{2}f^2 = \frac{1}{2}(2.875 + f)^2 = \frac{1}{2}(4.5 - 7.375 - f)^2$$

For squared error the approximation is exact, because the loss is already a parabola.

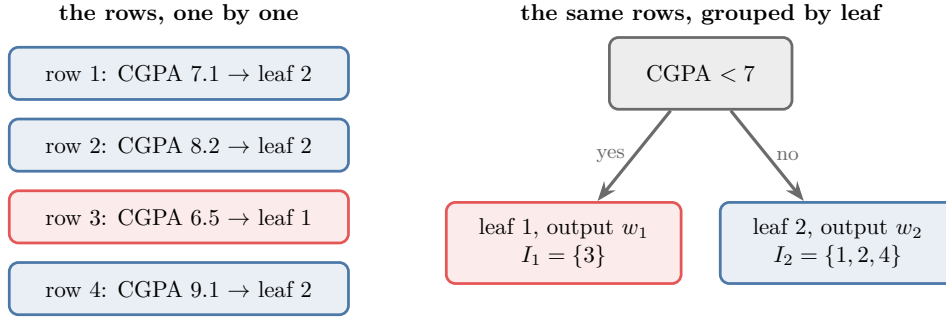
The first term, the old loss, does not depend on the new tree. It does not change where the minimum is, so we drop it:

$$\tilde{\text{Obj}}^{(t)} = \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

9. From a sum over rows to a sum over leaves

Key point: Every row in leaf j gets the same output w_j . So we can add up the rows leaf by leaf; with G_j and H_j the sums of g_i and h_i in leaf j , the objective becomes a sum of one parabola per leaf.

The objective now has two sums of different kinds: one over the n rows, one over the T leaves. To minimise it we bring them under one sum, over the leaves.



sum over the 4 rows = sum over the 2 leaves of the sum over the rows in that leaf; $f_t(x_1) = f_t(x_2) = f_t(x_4) = w_2$, $f_t(x_3) = w_1$

Figure 4: Four rows and a tree with two leaves: adding up the rows one by one gives the same total as adding them up leaf by leaf

Figure 4 shows the idea with a tree “CGPA < 7” and four rows with CGPA 7.1, 8.2, 6.5 and 9.1. Row 3 lands in leaf 1; rows 1, 2 and 4 land in leaf 2. The set of rows in leaf j is its **instance set** I_j : here $I_1 = \{3\}$ and $I_2 = \{1, 2, 4\}$.

Two facts make the regrouping work:

1. **The same rows, a different order.** Summing over all rows equals summing over the leaves, and inside each leaf over its rows: $\sum_{i=1}^4 = \sum_{i \in I_1} + \sum_{i \in I_2}$.
2. **Every row in a leaf gets that leaf’s output.** For $i \in I_j$, $f_t(x_i) = w_j$: here $f_t(x_3) = w_1$ and $f_t(x_1) = f_t(x_2) = f_t(x_4) = w_2$.

So, for example, the gradient part becomes $g_3 w_1 + (g_1 + g_2 + g_4) w_2$. Writing the sums of the gradients and Hessians in leaf j as

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i$$

and collecting the w_j^2 terms with the penalty:

1. **In words:** for each leaf, its gradient sum times its output, plus half of (its Hessian sum plus λ) times its output squared; add over the leaves, then add γT .
2. **Formula:**

$$\text{Obj}^{(t)} = \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T$$

3. **Example:** the leaf of the regression tree holding students 1 and 3 (residuals -2.875 and -1.375) has $g = 2.875$ and 1.375 , so $G = 4.25$ and $H = 1 + 1 = 2$. With $\lambda = 0$ its part of the objective is $4.25 w + w^2$.

Each leaf now has its own small parabola in its own w_j , independent of the other leaves.

10. The best leaf output

Key point: Set the derivative of each leaf's parabola to 0: $w_j^* = -G_j/(H_j + \lambda)$.

Differentiate $G_j w_j + \frac{1}{2}(H_j + \lambda)w_j^2$ with respect to w_j ; the other leaves and γT do not contain w_j and drop out.

$$\frac{\partial \text{Obj}}{\partial w_j} = G_j + (H_j + \lambda)w_j = 0$$

1. **In words:** the best output of a leaf is minus its gradient sum divided by (its Hessian sum plus λ).
2. **Formula:**

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

3. **Example:** the leaf of section 9, with $G = 4.25$, $H = 2$, $\lambda = 0$:

$$w^* = -\frac{4.25}{2 + 0} = -2.125$$

the output found in the [XGBoost regression Note](#), section 8. With $\lambda = 1$: $-4.25/3 = -1.42$.

The Notebook checks this against a brute-force search over a fine grid of w values: the minimum lands at the same place. Because $H_j + \lambda > 0$, the parabola opens upwards, so this point is a minimum, not a maximum.

11. Regression: gradient and Hessian of squared error

Key point: For $L = \frac{1}{2}(y - \hat{y})^2$: $g_i = \hat{y}_i - y_i = -r_i$ and $h_i = 1$. So $w^* = \sum r_i/(n + \lambda)$, the regression output formula.

1. **In words:** the gradient is minus the residual; the Hessian is 1 for every row.
2. **Formula:** with the chain rule,

$$g_i = \frac{\partial}{\partial \hat{y}_i} \frac{1}{2}(y_i - \hat{y}_i)^2 = -(y_i - \hat{y}_i) = -r_i, \quad h_i = \frac{\partial}{\partial \hat{y}_i} (\hat{y}_i - y_i) = 1$$

Here $\hat{y}_i$ is the previous stage's prediction, so r_i is the residual the new tree is fitted to. In a leaf with n rows, $G_j = -\sum r_i$ and $H_j = n$:

$$w_j^* = -\frac{\sum r_i}{n + \lambda} = \frac{\sum r_i}{n + \lambda}$$

3. **Example:** student 4 alone in a leaf, residual 0.625: $w^* = 0.625/(1 + 0) = 0.625$.

The $\frac{1}{2}$ in the loss is there only to cancel the 2 from the square, as in the [gradient boosting maths Note](#). XGBoost's `reg:squarederror` uses exactly this loss, with gradient $\hat{y} - y$ and Hessian 1.

12. Classification: gradient and Hessian of log loss

Key point: For log loss, measured in log-odds: $g_i = p_i - y_i = -r_i$ and $h_i = p_i(1 - p_i)$. So $w^* = \sum r_i / (\sum p_i(1 - p_i) + \lambda)$, the classification output formula.

In classification the model adds up log-odds, so the “prediction” we differentiate with respect to is the log-odds z . The two derivatives of log loss with respect to z are worked out in the [gradient boosting classification Note](#) (using the sigmoid derivative of the [sigmoid derivative Note](#)):

$$g_i = p_i - y_i, \quad h_i = p_i(1 - p_i)$$

with p_i the previous stage’s probability.

1. **In words:** the gradient is minus the residual in probability; the Hessian is $p(1 - p)$.
2. **Formula:**

$$w_j^* = -\frac{\sum (p_i - y_i)}{\sum p_i(1 - p_i) + \lambda} = \frac{\sum r_i}{\sum p_i(1 - p_i) + \lambda}$$

3. **Example:** the left leaf of the [XGBoost classification Note](#): classes 0, 1, 0, each with $p = 0.6$. $G = 0.6 - 0.4 + 0.6 = 0.8$ and $H = 3 \times 0.24 = 0.72$:

$$w^* = -\frac{0.8}{0.72 + 0} = -1.11$$

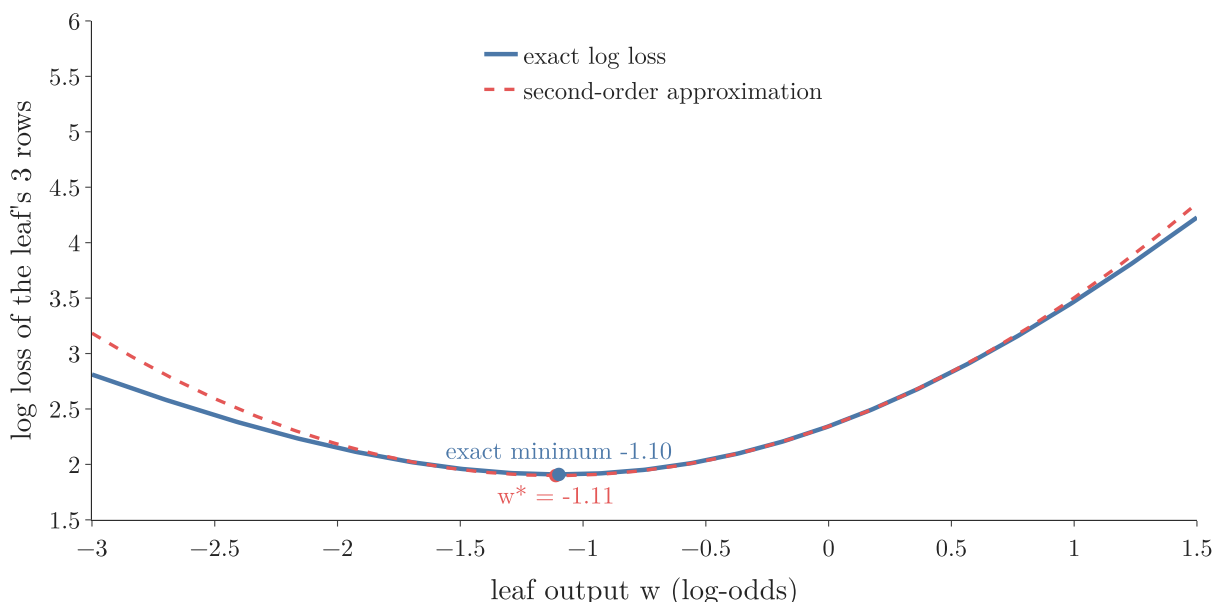


Figure 5: Exact log loss of this leaf’s three rows (blue) and its second-order approximation (red) as the leaf output w changes: they agree near 0, and their minima are close

For log loss the parabola is only an approximation (Figure 5). Its minimum, -1.11 , is close to the true minimum of the log loss, $\ln 0.5 - \ln 1.5 = -1.10$. The intuition: each new tree

starts again from the new predictions, so the small error is corrected rather than carried along. A Notebook test agrees: over 100 trees the error did not build up (Extra below).

Extra: The exact best output of a leaf solves $\sum_i \sigma(z_i + w) = \sum_i y_i$, where z_i is observation i 's previous log-odds (set the derivative of the leaf's log loss to 0). Here all three observations share one z_i , so the equation has a formula; in general it does not, and XGBoost uses the parabola instead. Does the parabola's small error build up? The Notebook (section 6) grows 100 trees of depth 3 ($\eta = 0.3$) on 2,000 synthetic observations, half for testing, twice with the same tree-growing rule: once with the Newton outputs $-G/H$, once with each leaf's exact minimum.

Trees	Train log loss, Newton	Train log loss, exact	Test log loss, Newton	Test log loss, exact
1	0.527	0.472	0.545	0.499
10	0.199	0.170	0.280	0.268
50	0.057	0.040	0.261	0.264
100	0.016	0.007	0.282	0.312

The Newton outputs lower the training loss a little less per tree, but the gap does not grow: it shrinks from 0.055 after one tree to 0.009 after 100. On the test half the two are about equal at 10 to 50 trees. So, on this data, the approximation does not build up into a large error.

Extra: This is Newton's method: a step to the minimum of the local parabola, using both the slope (g) and the curvature (h). scikit-learn's gradient boosting chooses its splits with the gradient alone and uses h only for the leaf outputs; XGBoost uses h for both the splits and the leaves (Sigrist 2021). The same g and h are what a custom loss must supply ([imbalanced data Note](#), section 9).

13. The best objective, and the similarity score

Key point: Put w_j^* back into the objective: the smallest objective a tree can reach is $-\frac{1}{2} \sum_j G_j^2 / (H_j + \lambda) + \gamma T$. The term $G_j^2 / (H_j + \lambda)$ is the similarity score.

Substituting $w_j^* = -G_j / (H_j + \lambda)$ into one leaf's part, $G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2$:

$$-\frac{G_j^2}{H_j + \lambda} + \frac{1}{2} (H_j + \lambda) \frac{G_j^2}{(H_j + \lambda)^2} = -\frac{G_j^2}{H_j + \lambda} + \frac{1}{2} \frac{G_j^2}{H_j + \lambda} = -\frac{1}{2} \frac{G_j^2}{H_j + \lambda}$$

1. **In words:** the best objective of a tree is minus a half of the sum over its leaves of (gradient sum squared over Hessian sum plus λ), plus γ per leaf.
2. **Formula:**

$$\text{Obj}^* = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

3. **Example:** the regression tree with one split at $\text{CGPA} < 8.25$, $\lambda = \gamma = 0$. Left: $G = 2.875 + 1.375 - 0.625 = 3.625$, $H = 3$. Right: $G = -3.625$, $H = 1$.

$$\text{Obj}^* = -\frac{1}{2} \left(\frac{3.625^2}{3} + \frac{3.625^2}{1} \right) = -\frac{1}{2}(4.38 + 13.14) = -8.76$$

The lower this number, the better the tree's structure. The paper uses it as a score for a tree, like the impurity of a decision tree but valid for a wider range of losses (Chen and Guestrin 2016, §2.2).

The term in each leaf, $G_j^2/(H_j + \lambda)$, is the **similarity score**. Since $G_j = -\sum r_i$, squaring removes the sign:

- regression ($h = 1$): $(\sum r_i)^2/(n + \lambda)$;
- classification ($h = p(1 - p)$): $(\sum r_i)^2/(\sum p_i(1 - p_i) + \lambda)$.

These are the two similarity formulas of the table in section 1.

14. The gain of a split

Key point: Gain = objective before the split – objective after it = $\frac{1}{2}[S_L + S_R - S_{\text{parent}}] - \gamma$. The regression and classification Notes drop the $\frac{1}{2}$ and compare with γ separately, which is exactly what the XGBoost library does.

A split turns one leaf (the parent, with rows I) into two (left I_L and right I_R). The number of leaves grows by 1, so the penalty grows by γ .

- Before: $-\frac{1}{2}G^2/(H + \lambda) + \gamma$, with $G = G_L + G_R$ and $H = H_L + H_R$.
- After: $-\frac{1}{2}(G_L^2/(H_L + \lambda) + G_R^2/(H_R + \lambda)) + 2\gamma$.

1. **In words:** the gain is how much the split lowers the objective: half of (left similarity + right similarity – parent similarity), minus γ for the extra leaf.
2. **Formula:**

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

3. **Example:** the root split of the regression tree, $\lambda = \gamma = 0$ (parent similarity 0, section 13):

$$\text{Gain} = \frac{1}{2} (4.38 + 13.14 - 0) - 0 = 8.76$$

which is $0 - (-8.76)$, the drop in the best objective.

The split is worth making only if the gain is positive. With $\gamma > 0$, a split must lower the loss by more than the price of one extra leaf: this is the pruning rule of the [XGBoost regression Note](#), section 13.

The regression and classification Notes used $S_L + S_R - S_{\text{parent}}$, without the $\frac{1}{2}$, and got 17.52 for this split. The $\frac{1}{2}$ only rescales every gain by the same factor, so the best split is the same either way.

The XGBoost library follows the same convention as those Notes. Its tree dump prints `gain=17.520834` for this split, and it removes a split when that unhalved gain is below `gamma`: in the Notebook the lower split (gain 5.04) survives `gamma=5.0` and disappears at `gamma=5.1`. So the library's `gamma` corresponds to 2γ in the paper's formula.

15. Summary

Step	Result
Objective at stage t	$\sum_i L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \gamma T + \frac{1}{2}\lambda \sum_j w_j^2$
Taylor, second order	$\sum_i [g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2] + \Omega(f_t)$
Grouped by leaf	$\sum_j [G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2] + \gamma T$
Best leaf output	$w_j^* = -G_j / (H_j + \lambda)$
Best objective	$-\frac{1}{2} \sum_j G_j^2 / (H_j + \lambda) + \gamma T$
Gain of a split	$\frac{1}{2}[S_L + S_R - S_P] - \gamma$, with $S = G^2 / (H + \lambda)$
Squared error	$g = -r$, $h = 1$
Log loss (in log-odds)	$g = p - y$, $h = p(1 - p)$

- XGBoost minimises loss plus a penalty on the number of leaves (γ) and on the size of their outputs (λ).
- A second-order Taylor series turns any loss into one parabola per leaf, which is minimised in closed form.
- The leaf output and the similarity score both come from that minimisation; only g and h depend on the loss.
- The gain is the drop in the best objective after a split; the library drops the $\frac{1}{2}$ and compares with `gamma`.

16. Sources

- Chen, T. and Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD 2016 (arXiv:1603.02754).
- Sigrist, F. (2021). *Gradient and Newton Boosting for Classification and Regression*. Expert Systems with Applications (arXiv:1808.03064), section on software implementations.

17. Key terms

Term	Meaning
Objective function	The quantity a model minimises: loss plus regularisation term
Regularisation term Ω	XGBoost's penalty on a tree: $\gamma T + \frac{1}{2}\lambda \sum_j w_j^2$
Leaf weight w_j	The output value of leaf j of a tree
Taylor series	Approximation of a function near a point by a polynomial built from its derivatives there
Gradient g_i	First derivative of row i 's loss with respect to the previous prediction
Hessian h_i	Second derivative of row i 's loss with respect to the previous prediction
Instance set I_j	The rows that land in leaf j

Term	Meaning
Structure score	The best objective of a tree, $-\frac{1}{2} \sum_j G_j^2 / (H_j + \lambda) + \gamma T$; lower is better